Academic Session 2025-26

EVEN Semester Jan-Jun 2026

UNIVERSITY INSTITUTE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BTECH CSE – CS 201

SEMESTER: 6TH

CLOUD COMPUTING

(23CSH-307)

Unit No. 1 **Chapter No. 2** **Lecture No. 5**

Topic : NIST Cloud Architecture

Sanjay Tiwari, E18823 **Senior Technical Trainer**

Learning Objectives

- Understand the purpose and structure of the NIST Cloud Reference Architecture.
- Learn the roles of cloud consumer, provider, broker, carrier, and auditor.
- Explore how NIST components interact to form a complete cloud ecosystem.
- Identify common architectural patterns based on NIST’s conceptual framework.

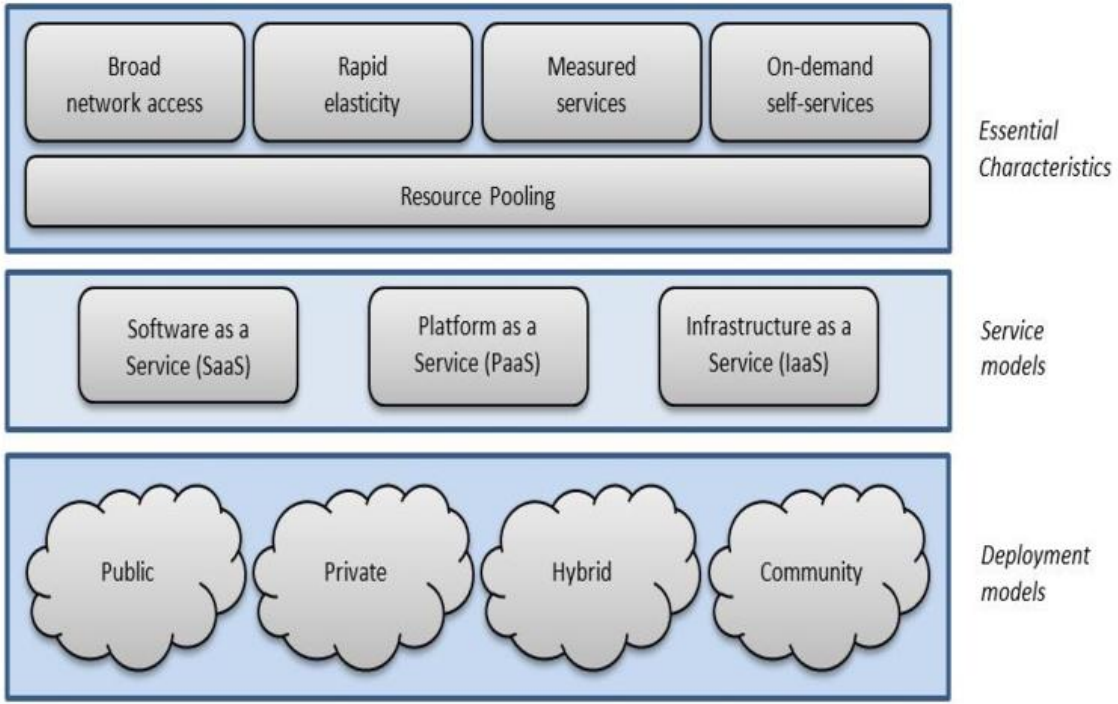
NIST Cloud Reference Architecture: Overview

- Serves as a standardized framework for understanding cloud ecosystems.
- Defines the key actors and their roles within cloud service environments.
- Focuses on interactions, responsibilities, and operational boundaries.
- Provides a consistent vocabulary for cloud discussions and documentation.
- Helps organizations evaluate cloud systems using a reference structure.
- Represents cloud systems as interconnected service providers and consumers.
- Includes service orchestration, management, monitoring, and governance layers.
- Supports high-level design decisions for enterprise cloud adoption.

Core Components Defined by NIST

- **Cloud Consumer** – uses cloud services and manages usage/controls.
- **Cloud Provider** – delivers cloud services and manages infrastructure.
- **Cloud Broker** – negotiates, aggregates, or arbitrates cloud services.
- **Cloud Carrier** – ensures network connectivity and data transport.
- **Cloud Auditor** – evaluates services for compliance and governance.
- These components interact to ensure reliable service delivery.
- Together they define operational boundaries in cloud ecosystems.
- Allow organizations to map responsibilities clearly across stakeholders.

NIST Cloud Architecture



Source: https://www.researchgate.net/figure/NIST-Visual-model-of-cloud-computing-definition_fig1_299982137

Cloud Consumer (NIST Role)

- Entity that uses cloud services delivered by a provider.
- Responsible for managing user access, application usage, and service subscriptions.
- Requests, configures, and manages resources as allowed by the service.
- Monitors service consumption, performance, and behavior.
- Ensures proper governance of cloud usage within the organization.
- Interacts directly with dashboards, APIs, or management consoles.
- May rely on brokers for service selection or optimization.
- Plays a key role in defining service requirements and consumption patterns.

Cloud Provider (NIST Role)

- The entity responsible for offering cloud services to consumers.
- Manages infrastructure, platforms, or software required to deliver services.
- Handles provisioning, monitoring, and operational tasks.
- Ensures availability, security, and performance of services.
- Controls internal workflows such as orchestration and resource scheduling.
- Provides tools, APIs, and interfaces for consumer usage.
- Establishes SLAs that govern service delivery expectations.
- Coordinates with carriers, brokers, and auditors as needed.

Cloud Broker (NIST Role)

- Facilitates use, performance, and delivery of cloud services.
- Acts as an intermediary between consumer and provider.
- Can aggregate services from multiple providers into a single offering.
- Performs service arbitration to match consumer needs with provider offerings.
- Supports workload distribution across various cloud services.
- May provide cost optimization, usage analytics, or service management tools.
- Allows consumers to access curated cloud solutions rather than raw offerings.
- Helps organizations simplify multi-cloud management strategies.

Cloud Carrier (NIST Role)

- Provides network connectivity between consumer and provider.
- Responsible for secure and reliable data transport.
- Ensures stable communication channels for service access.
- May use dedicated networking, VPNs, or public internet routes.
- Coordinates routing, bandwidth allocation, and traffic flow.
- Plays a major role in performance and latency of cloud interactions.
- Ensures continuity in communication via redundant network paths.
- Collaborates with providers to maintain connection quality and uptime.

Cloud Auditor (NIST Role)

- Independent party that assesses cloud services for compliance.
- Evaluates performance of cloud providers against internal/external standards.
- Performs security audits, operational checks, and governance reviews.
- Ensures cloud environments meet regulatory requirements.
- Reports findings to stakeholders for transparency and assurance.
- Validates adherence to SLAs and security frameworks.
- Helps organizations build trust in cloud adoption.
- Can be internal or third-party depending on the audit scope.

NIST Architectural Patterns (Allowed Patterns Only)

- **Service Orchestration Pattern** – coordinated management of cloud services.
- **Intermediation Pattern** – broker enhances service capabilities for consumers.
- **Aggregation Pattern** – combining multiple services into a unified solution.
- **Arbitrage Pattern** – selecting services dynamically based on criteria such as cost.
- **Distributed Cloud Access Pattern** – using carriers to route services across locations.
- **Audit-Assisted Governance Pattern** – leveraging auditors for compliance operations.
- **Multi-role Interaction Pattern** – defines communication between consumer, provider, broker, carrier, auditor.
- Patterns guide how components collaborate and interact in cloud ecosystems.

Component Interactions in NIST Architecture

- Consumer requests services; provider provisions them accordingly.
- Broker may assist consumer by optimizing or customizing services.
- Carrier manages data flow and communication pathways.
- Auditor evaluates provider operations for compliance and transparency.
- Provider and carrier collaborate on network reliability and uptime.
- Broker and provider work together for aggregated or enhanced offerings.
- Consumer relies on carrier for smooth service connectivity.
- All roles interact dynamically to complete the cloud service lifecycle.

Summary of the Lecture

- NIST Cloud Architecture defines five essential cloud ecosystem roles.
- These roles clarify operational boundaries and interactions.
- Cloud Consumer, Provider, Broker, Carrier, Auditor work as an integrated system.
- Architectural patterns illustrate how processes and roles coordinate.
- The framework supports governance, transparency, and cloud adoption structure.
- Organizations use it to map workflows and responsibilities clearly.
- It standardizes cloud ecosystem understanding across industries.

Faculty-curated videos, NPTEL, Coursera, LinkedIn, or other relevant learning resources

- Cloud Computing: https://onlinecourses.nptel.ac.in/noc21_cs14/preview
- Cloud Computing: https://onlinecourses.nptel.ac.in/noc26_cs55/preview
- Google Cloud Computing Foundations: https://onlinecourses.nptel.ac.in/noc20_cs55/preview
- Google Cloud Computing Foundations: https://onlinecourses.nptel.ac.in/noc24_cs131/preview

Next Lecture

Vertical scaling vs horizontal scaling, elasticity concepts, scalability design patterns and best practices

Quiz/ FAQ's

1. Why does NIST define cloud roles?

To standardize communication, clarify responsibilities, and ensure consistent understanding across cloud ecosystems.

2. Is a cloud broker mandatory in all implementations?

No — brokers are optional and used when aggregation or optimization is required.

3. Does a cloud carrier manage cloud resources?

No — it only handles connectivity and data transport.

4. Who validates compliance in cloud environments?

The cloud auditor evaluates compliance, security, and governance.

5. Can one organization play multiple NIST roles?

Yes — for example, a provider can also act as a broker.

References/ Articles/ Videos

- NIST Cloud Computing Reference Architecture (Full PDF)
  https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-145.pdf
- NIST Cloud Computing Standards Roadmap
  https://www.nist.gov/publications/nist-cloud-computing-standards-roadmap
- NIST Cloud Computing Synopsis and Recommendations
  https://www.nist.gov/publications/nist-cloud-computing-synopsis-and-recommendations
- NIST Service Interoperability Standards
  https://www.nist.gov/programs-projects/cloud-computing
- IEEE — Cloud Architecture Studies
  https://ieeexplore.ieee.org
  IBM Cloud Reference Model Overview
  https://www.ibm.com/cloud/architecture
  Red Hat — Cloud Architecture Concepts
  https://www.redhat.com/en/topics/cloud
- Cloud Security Alliance — Cloud Governance Materials
  https://cloudsecurityalliance.org/

.

Faculty-curated videos, NPTEL, Coursera, LinkedIn, or other relevant learning resources

- Cloud Computing: https://onlinecourses.nptel.ac.in/noc21_cs14/preview
- Cloud Computing: https://onlinecourses.nptel.ac.in/noc26_cs55/preview
- Google Cloud Computing Foundations: https://onlinecourses.nptel.ac.in/noc20_cs55/preview
- Google Cloud Computing Foundations: https://onlinecourses.nptel.ac.in/noc24_cs131/preview

Class-Wise Feedback

Thank You

For queries
Email: sanjay.e18823@cumail.in

Academic Session 2025-26

EVEN Semester Jan-Jun 2026

UNIVERSITY INSTITUTE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BTECH CSE – CS 201

SEMESTER: 6TH

CLOUD COMPUTING

(23CSH-307)

Unit No. 1 Chapter No. 2 Lecture No. 6

Topic : Vertical scaling vs horizontal scaling, elasticity concepts, scalability design patterns

Sanjay Tiwari, E18823 **Senior Technical Trainer**

RECAP

- NIST Cloud Architecture defines five essential roles within the cloud ecosystem.
- These roles help clarify operational boundaries and responsibilities.
- The Cloud Consumer represents the end user of cloud services.
- The Cloud Provider delivers and manages cloud services and infrastructure.
- The Cloud Broker facilitates service use, integration, and optimization.
- The Cloud Carrier provides connectivity and transport between parties.
- The Cloud Auditor ensures compliance, security, and performance assurance.
- Architectural patterns explain coordination among roles and processes.
- The framework promotes governance, transparency, and structured cloud adoption.
- NIST architecture standardizes cloud ecosystem understanding across industries.

Learning Objectives

- Distinguish between vertical and horizontal scaling
- Understand elasticity and its role in dynamic systems
- Identify scalability design patterns
- Apply best practices for scalable architectures

Vertical Scaling (Scale Up)

- Increases capacity of a single system
- Achieved by upgrading CPU, memory, or storage
- Maintains a centralized system structure
- Requires minimal application-level changes
- Suitable for tightly coupled applications
- Performance improves linearly with upgrades
- Hardware limits restrict maximum scalability
- Downtime may be required during upgrades

Advantages of Vertical Scaling

- Simple implementation approach
- No need for distributed system redesign
- Easier troubleshooting and monitoring
- Lower architectural complexity
- Predictable performance gains
- Suitable for legacy systems
- Reduced coordination overhead
- Centralized resource management

Limitations of Vertical Scaling

- Physical hardware imposes scaling limits
- Single point of failure remains
- Cost increases sharply with upgrades
- Limited fault tolerance
- Downtime risks during scaling
- Not suitable for unpredictable workloads
- Vendor hardware dependency
- Inefficient for large-scale growth

Horizontal Scaling (Scale Out)

- Adds multiple resource instances
- Distributes workload across nodes
- Enables parallel processing
- Improves system availability
- Supports incremental capacity expansion
- Requires distributed system design
- Works well with modular applications
- Facilitates fault isolation

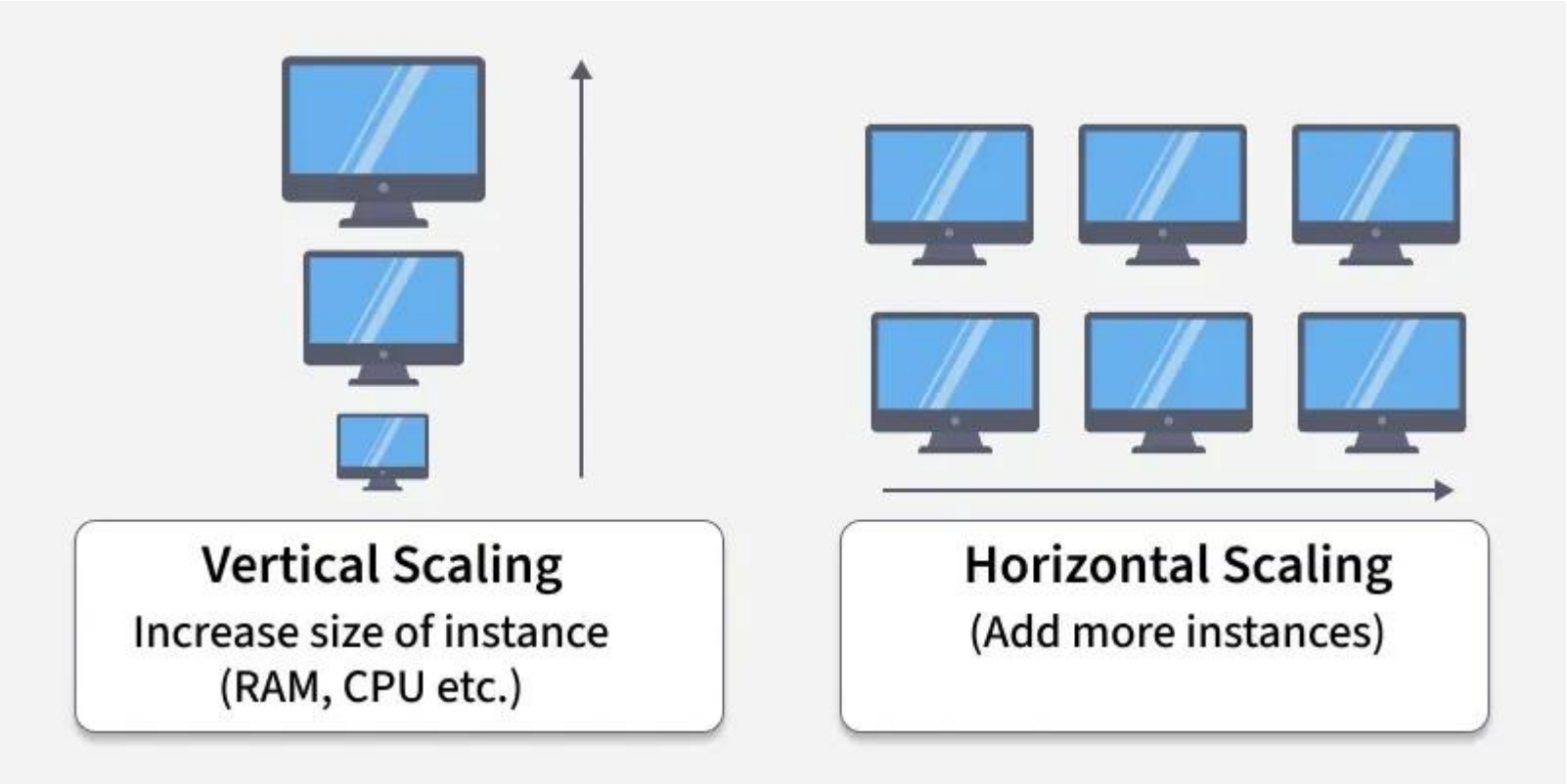
Advantages of Horizontal Scaling

- Near-unlimited scalability potential
- Improved fault tolerance
- High availability through redundancy
- Better workload distribution
- Cost-effective incremental growth
- Supports large user bases
- Enables rolling updates
- Enhances system resilience

Challenges of Horizontal Scaling

- Increased architectural complexity
- Requires coordination between nodes
- Data consistency management needed
- Latency may increase due to distribution
- Monitoring becomes more complex
- Debugging is more challenging
- Requires careful system design
- Dependency management is critical

Comparison



Source: geeksforgeeks

Vertical vs Horizontal Scaling

- Vertical scaling enhances single-node capacity
- Horizontal scaling increases number of nodes
- Vertical scaling is hardware-bound
- Horizontal scaling supports redundancy
- Vertical scaling is simpler to implement
- Horizontal scaling improves fault tolerance
- Vertical scaling suits stable workloads
- Horizontal scaling suits dynamic workloads

Elasticity: Core Concepts

- Dynamic adjustment of resources
- Responds to real-time workload changes
- Enables automatic resource expansion
- Enables automatic resource reduction
- Optimizes performance efficiency
- Minimizes resource wastage
- Supports fluctuating demand patterns
- Enhances operational flexibility

Elasticity vs Scalability

- Scalability defines growth capability
- Elasticity defines responsiveness speed
- Scalability may be manual or static
- Elasticity emphasizes automation
- Scalability supports long-term growth
- Elasticity supports short-term fluctuations
- Elastic systems reduce over-provisioning
- Both concepts complement each other

Scalability Design Patterns

- Stateless service design
- Data partitioning techniques
- Caching mechanisms
- Asynchronous processing models
- Loose coupling of components
- Modular service architecture
- Parallel processing strategies
- Resource isolation approaches

Stateless Design Pattern

- No dependency on local session state
- Requests handled independently
- Simplifies horizontal scaling
- Enables easy instance replacement
- Improves fault tolerance
- Externalizes state storage
- Supports load distribution
- Enhances scalability efficiency

Data Partitioning Pattern

- Divides large datasets logically
- Distributes data across nodes
- Improves access performance
- Reduces system bottlenecks
- Enables parallel data processing
- Enhances scalability of storage
- Requires partition strategy planning
- Balances data load evenly

Caching Pattern

- Stores frequently accessed data
- Reduces repeated data retrieval
- Improves system response time
- Decreases backend load
- Enhances scalability efficiency
- Supports high-read workloads
- Requires cache consistency handling
- Optimizes resource utilization

Asynchronous Processing Pattern

- Separates request and processing flow
- Improves system responsiveness
- Handles heavy workloads efficiently
- Reduces blocking operations
- Enhances throughput capacity
- Enables background task execution
- Improves scalability under load
- Supports decoupled system design

Scalability Best Practices

- Design systems for growth
- Prefer stateless components
- Avoid tight component coupling
- Monitor performance continuously
- Identify scalability bottlenecks early
- Test systems under load
- Plan for failure scenarios
- Document scalability assumptions

Operational Best Practices

- Implement proactive monitoring
- Review capacity usage regularly
- Plan scaling thresholds carefully
- Optimize resource utilization
- Perform scalability testing periodically
- Analyze workload patterns
- Improve system observability
- Continuously refine architecture

Summary of the Lecture

- Vertical scaling increases single-system capacity
- Horizontal scaling increases system breadth
- Elasticity enables dynamic resource adjustment
- Scalability supports long-term growth
- Design patterns enable efficient scaling
- Best practices reduce scaling risks
- Proper design improves reliability
- Scalable systems adapt to demand

Next Lecture

Load balancing fundamentals, algorithms (round-robin, least connection, IP hash, weighted), auto-scaling mechanisms and triggers

Faculty-curated videos, NPTEL, Coursera, LinkedIn, or other relevant learning resources

- Cloud Computing: https://onlinecourses.nptel.ac.in/noc21_cs14/preview
- Cloud Computing: https://onlinecourses.nptel.ac.in/noc26_cs55/preview
- Google Cloud Computing Foundations: https://onlinecourses.nptel.ac.in/noc20_cs55/preview
- Google Cloud Computing Foundations: https://onlinecourses.nptel.ac.in/noc24_cs131/preview

Quiz/ FAQ's

- Q1. Is horizontal scaling always better than vertical scaling?
- Solution: No. Horizontal scaling suits large, dynamic workloads, while vertical scaling is effective for stable, smaller systems.
- Q2. Can a system use both vertical and horizontal scaling together?
- Solution: Yes. Many systems first scale vertically and then horizontally when hardware limits are reached.
- Q3. What problem does elasticity primarily solve?
- Solution: Elasticity addresses fluctuating demand by automatically adjusting resources in real time.
- Q4. Is elasticity useful if workload is constant?
- Solution: Limited benefit. Elasticity is most effective when demand varies significantly.
- Q5. Why does horizontal scaling require careful design?
- Solution: Distributed systems introduce complexity in coordination, data handling, and monitoring.

References/ Articles/ Videos

https://martinfowler.com/articles/scaling.html
https://aws.amazon.com/what-is/scalability/
https://cloud.google.com/architecture/scalable-and-resilient-apps
https://learn.microsoft.com/azure/architecture/guide/design-principles/scalability
https://www.redhat.com/en/topics/cloud-native-apps/scalability
https://www.ibm.com/topics/scalability
https://www.digitalocean.com/community/tutorials
https://developer.mozilla.org/en-US/docs/Web

Class-Wise Feedback

Thank You

For queries
Email: sanjay.e18823@cumail.in

Academic Session 2025-26

EVEN Semester Jan-Jun 2026

UNIVERSITY INSTITUTE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BTECH CSE – CS 201

SEMESTER: 6TH

CLOUD COMPUTING

(23CSH-307)

Unit No. 1 **Chapter No. 2** **Lecture No. 7**

Topic : Load balancing fundamentals, algorithms and auto scaling mechanisms

Sanjay Tiwari, E18823 **Senior Technical Trainer**

Learning Objectives

- Understand load balancing fundamentals
- Compare common load balancing algorithms
- Explain auto-scaling mechanisms
- Identify auto-scaling triggers and behaviors

Introduction to Scalability in Computing Systems

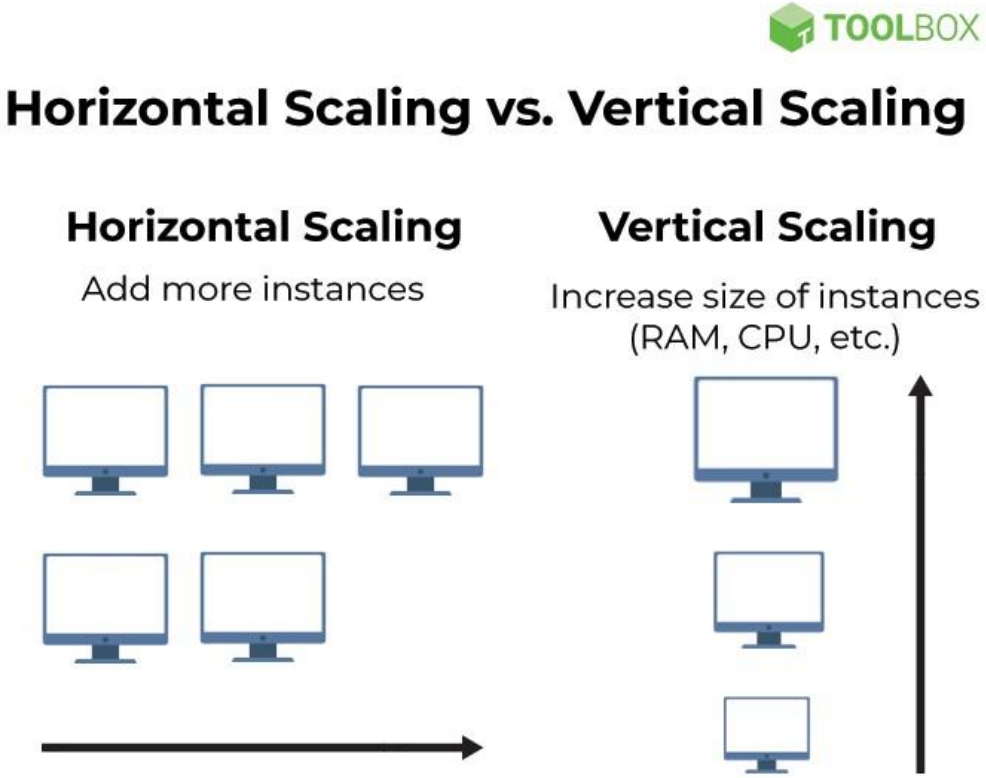
- Scalability refers to a system’s ability to handle increasing workload efficiently
- It ensures consistent performance as demand, users, or data volume grows
- Scalability is a critical requirement for modern distributed and cloud systems
- It supports long-term business growth without frequent system redesign
- Scalability helps reduce performance bottlenecks under peak usage
- It improves system reliability and user satisfaction
- Scalable systems adapt to changing resource requirements
- Scalability is closely linked to cost efficiency and resource optimization

Importance of Scalability

- Scalability allows applications to support growing user bases
- It prevents system crashes during traffic spikes
- It ensures stable response time and throughput
- Scalability improves return on infrastructure investment
- It supports global availability and expansion
- It reduces the need for frequent hardware replacement
- Scalability enables seamless integration of new services
- It enhances system resilience and fault tolerance

Types of Scalability

- Scalability can be achieved using multiple architectural approaches
- Vertical scalability focuses on increasing individual resource capacity
- Horizontal scalability focuses on increasing the number of resources
- Each type has unique benefits and limitations
- Choice of scalability depends on application requirements
- Infrastructure design strongly influences scalability type
- Scalability impacts system cost and performance
- Understanding scalability types is essential for system architects



Source: Toolbox

Vertical Scalability Overview

- Vertical scalability is also known as scaling up
- It involves adding more power to a single machine
- Resources upgraded include CPU, RAM, and storage
- It improves performance by strengthening existing hardware
- Vertical scaling maintains a single system image
- It is easier to implement than distributed scaling
- Applications often require minimal changes
- Vertical scalability has physical hardware limitations

Characteristics of Vertical Scalability

- Performance improvement is immediate after resource upgrade
- System architecture remains largely unchanged
- Data consistency management is simpler
- Vertical scaling supports legacy applications
- Downtime may be required during upgrades
- Resource upgrades have an upper limit
- Costs increase significantly at higher capacities
- Single point of failure remains a concern

Advantages of Vertical Scalability

- Simple implementation with minimal configuration changes
- No need for complex distributed system management
- Lower software complexity compared to distributed systems
- Easier monitoring and maintenance
- Strong performance improvement for compute-intensive tasks
- Suitable for databases with strict consistency requirements
- Faster deployment compared to horizontal scaling
- Reduced network communication overhead

Limitations of Vertical Scalability

- Hardware upgrades are limited by machine capacity
- Scaling beyond a certain point becomes impossible
- High-end hardware significantly increases cost
- Downtime may impact availability during upgrades
- Single system failure affects entire application
- Not suitable for massive user growth
- Limited fault tolerance compared to distributed systems
- Less flexible for dynamic workload changes

Horizontal Scalability Overview

- Horizontal scalability is also known as scaling out
- It involves adding more machines or instances
- Workload is distributed across multiple systems
- It supports large-scale distributed applications
- Horizontal scaling increases system redundancy
- It enables handling of massive traffic volumes
- Systems are designed to work as a cluster
- Horizontal scalability improves availability and reliability

Characteristics of Horizontal Scalability

- Resources grow by adding identical or similar nodes
- System performance improves with each added node
- Workload is shared among multiple machines
- System architecture is inherently distributed
- Fault tolerance is higher due to redundancy
- Scaling can occur without system downtime
- Data management becomes more complex
- Network communication plays a critical role

Advantages of Horizontal Scalability

- Virtually unlimited scaling potential
- High availability through redundancy
- Better fault isolation and recovery
- Cost efficiency using commodity hardware
- Supports global user distribution
- Enables parallel processing of requests
- Suitable for modern web-scale applications
- Flexible resource expansion over time

Challenges of Horizontal Scalability

- Increased system design complexity
- Data consistency becomes harder to maintain
- Requires advanced monitoring and coordination
- Network latency affects performance
- Application must be designed for distribution
- Debugging distributed failures is difficult
- Security management becomes more complex
- Requires effective traffic distribution mechanisms

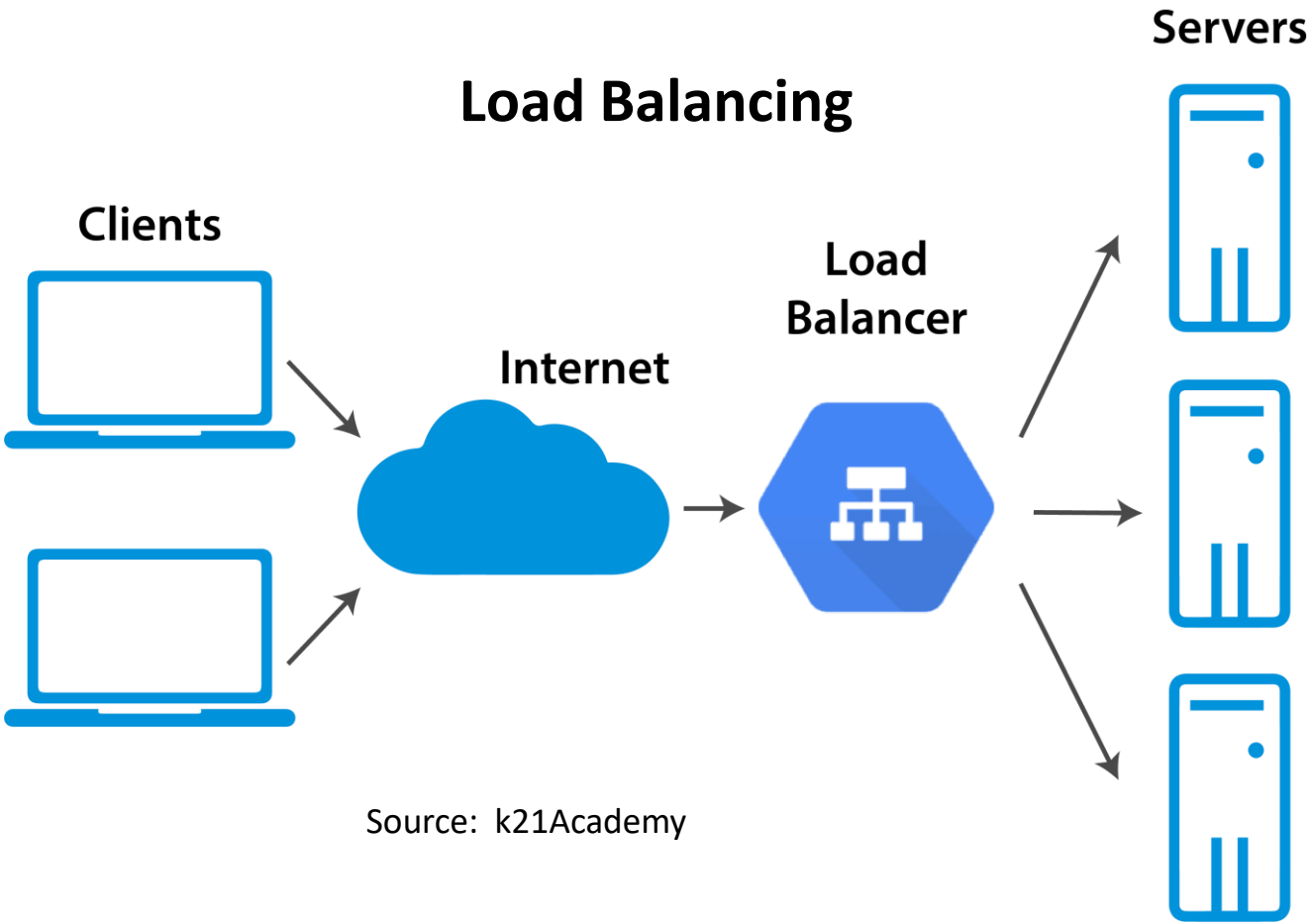
Scalability Comparison: Vertical vs Horizontal

- Vertical scaling strengthens a single machine
- Horizontal scaling increases the number of machines
- Vertical scaling has physical limitations
- Horizontal scaling supports large-scale growth
- Vertical scaling is simpler to manage
- Horizontal scaling provides better fault tolerance
- Cost behavior differs significantly between approaches
- Modern systems often prefer horizontal scalability

Load Balancing Fundamentals

- Distributes incoming client requests across multiple resources
- Prevents any single component from becoming a bottleneck
- Improves overall system responsiveness and throughput
- Enhances service availability during high traffic
- Supports fault tolerance by redirecting failed requests
- Enables efficient utilization of backend resources
- Acts as an intermediary between clients and services
- Forms the foundation for scalable system design

Load Balancing



Source: k21Academy

Objectives of Load Balancing

- Maintain consistent application performance under load
- Minimize response time during traffic surges
- Ensure high availability through redundancy
- Reduce risk of system-wide failures
- Improve user experience across peak usage
- Enable smooth horizontal expansion
- Support service reliability goals
- Simplify traffic management

Round-Robin Algorithm

- Routes requests sequentially across available servers
- Ensures equal distribution without considering load
- Simple logic with minimal computational overhead
- Effective when servers have identical capacity
- Does not adapt to runtime performance variations
- May overload slower or busy servers
- Best suited for uniform request patterns
- Commonly used for basic load distribution

Least Connection Algorithm

- Routes traffic to server with fewest active connections
- Dynamically adapts to current server load
- Suitable for long-running or variable requests
- Provides better balancing than round-robin
- Requires continuous connection tracking
- Slightly higher processing overhead
- Handles heterogeneous workloads efficiently
- Improves fairness in request distribution

IP Hash Algorithm

- Uses client IP address to select server
- Ensures request consistency for same client
- Supports session persistence naturally
- Reduces need for centralized session storage
- Can lead to uneven traffic distribution
- Sensitive to uneven IP address patterns
- Scaling changes may disrupt mapping
- Useful for state-dependent applications

Weighted Load Balancing

- Assigns traffic proportionally based on server capacity
- Allows stronger servers to handle more requests
- Supports heterogeneous infrastructure environments
- Enhances overall performance efficiency
- Requires accurate capacity estimation
- Weights must be reviewed periodically
- Can be combined with other algorithms
- Optimizes resource utilization strategically

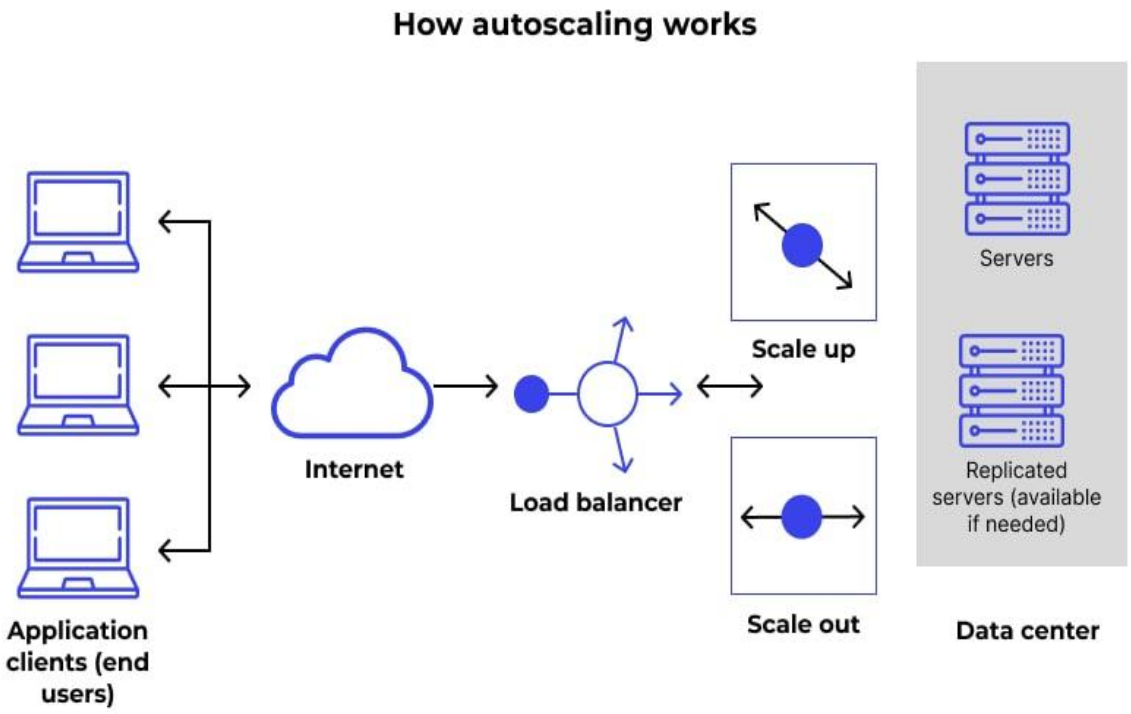
Auto-Scaling Fundamentals

- Automatically adjusts number of active resources
- Matches system capacity with real-time demand
- Reduces manual infrastructure management
- Helps control operational costs
- Improves application reliability during spikes
- Supports both scaling out and scaling in
- Enables elastic behavior under varying load
- Works alongside load balancing mechanisms

Auto-Scaling Triggers

- CPU utilization exceeding defined thresholds
- Memory consumption reaching critical levels
- Increase in request or transaction rate
- Rising response time beyond acceptable limits
- Growth in task or message queue length
- Error rate exceeding tolerance levels
- Custom application-level metrics
- Time-based or calendar-based schedules

Auto-Scaling



Source: Wallarm

Load Balancing & Auto-Scaling Relationship

- Load balancer distributes traffic efficiently
- Auto-scaling adjusts available capacity dynamically
- New instances are automatically registered
- Load balancer routes traffic to scaled resources
- Together improve fault tolerance and resilience
- Enable seamless handling of traffic spikes
- Reduce downtime and performance degradation
- Form a core elasticity strategy

Summary of the Lecture

- Load balancing distributes client requests efficiently
- Algorithms define how traffic is routed
- Each algorithm fits specific workload patterns
- Auto-scaling adjusts capacity dynamically
- Triggers initiate scaling actions automatically
- Load balancing and scaling work together
- Improve system resilience and availability
- Essential for handling variable workloads

Next Lecture

Need for virtualization in cloud environments, key benefits (resource optimization, isolation, portability), virtualization overview

Quiz/ FAQ's

- Q1. Why is load balancing essential in scalable systems?
- Solution: It prevents overload, ensures availability, and maintains performance consistency.
- Q2. Why is round-robin not always effective?
- Solution: It ignores real-time server load, which can cause imbalance.
- Q3. When is least connection preferred?
- Solution: For workloads with variable or long-running requests.
- Q4. What advantage does IP hash provide?
- Solution: It maintains session consistency without external state storage.
- Q5. Why are weights used in load balancing?
- Solution: To account for differences in server capacity and performance.

Faculty-curated videos, NPTEL, Coursera, LinkedIn, or other relevant learning resources

- Cloud Computing: https://onlinecourses.nptel.ac.in/noc21_cs14/preview
- Cloud Computing: https://onlinecourses.nptel.ac.in/noc26_cs55/preview
- Google Cloud Computing Foundations: https://onlinecourses.nptel.ac.in/noc20_cs55/preview
- Google Cloud Computing Foundations: https://onlinecourses.nptel.ac.in/noc24_cs131/preview

References/ Articles/ Videos

- https://aws.amazon.com/what-is/load-balancing/
- https://cloud.google.com/architecture/load-balancing-overview
- https://learn.microsoft.com/azure/architecture/guide/technology-choices/load-balancing-overview
- https://www.nginx.com/resources/glossary/load-balancing/
- https://www.redhat.com/en/topics/cloud-native-apps/what-is-load-balancing
- https://aws.amazon.com/autoscaling/
- https://cloud.google.com/docs/autoscaler
- https://www.ibm.com/topics/load-balancing

Class-Wise Feedback

Thank You

For queries
Email: sanjay.e18823@cumail.in

Academic Session 2025-26

EVEN Semester Jan-Jun 2026

UNIVERSITY INSTITUTE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BTECH CSE – CS 201

SEMESTER: 6TH

CLOUD COMPUTING

(23CSH-307)

Unit No. 1 **Chapter No. 3** **Lecture No. 8**

Topic : Need for virtualization in cloud environments, key benefits (resource optimization, isolation, portability), virtualization overview

Sanjay Tiwari, E18823 **Senior Technical Trainer**

RECAP

- Load balancing distributes incoming client requests across multiple resources.
- Traffic routing is governed by specific load balancing algorithms.
- Different algorithms are suited to different workload characteristics.
- Auto-scaling dynamically adjusts resource capacity based on demand.
- Scaling actions are initiated through predefined triggers and policies.
- Load balancing and auto-scaling operate together to optimize performance.
- These mechanisms enhance system resilience and fault tolerance.
- High availability is achieved by avoiding single points of failure.
- They enable efficient handling of fluctuating and unpredictable workloads.
- Load balancing and scaling are essential for modern cloud architectures.

Learning Objectives

- Understand the concept of virtualization
- Explain the need for virtualization in cloud environments
- Identify key benefits of virtualization
- Relate virtualization to efficient cloud operations

Virtualization Overview

- Abstracts physical hardware into virtual resources
- Enables multiple logical environments on one machine
- Separates software from hardware dependency
- Allows flexible resource allocation
- Improves overall infrastructure efficiency
- Enables rapid environment creation
- Simplifies infrastructure management
- Acts as a foundation for cloud infrastructure

Virtualization: Core Working Concept

- Physical resources are shared logically
- Each virtual environment runs independently
- A virtualization layer manages hardware access
- Resources are allocated dynamically
- Multiple workloads coexist securely
- Improves utilization of idle hardware
- Reduces dependency on physical systems
- Enables scalable infrastructure design

Need for Virtualization in Cloud Environments

- Cloud demands efficient use of hardware resources
- Traditional systems leave resources underutilized
- Virtualization enables resource consolidation
- Supports on-demand infrastructure provisioning
- Enables rapid service deployment
- Allows multiple users on shared infrastructure
- Improves scalability readiness
- Enhances infrastructure flexibility

Operational Need for Virtualization

- Enables dynamic allocation of resources
- Supports multi-tenant cloud environments
- Simplifies infrastructure expansion
- Reduces manual hardware configuration
- Improves workload management
- Enhances disaster recovery capability
- Enables faster system recovery
- Improves overall operational agility

Key Benefit: Resource Optimization

- Maximizes CPU utilization
- Improves memory usage efficiency
- Enables storage consolidation
- Reduces idle physical resources
- Supports workload consolidation
- Lowers infrastructure costs
- Improves energy efficiency
- Reduces hardware footprint

Key Benefit: Isolation

- Separates workloads securely
- Prevents interference between environments
- Improves system stability
- Limits failure impact to single environment
- Enhances security boundaries
- Supports safe multi-tenant usage
- Enables independent configuration
- Improves fault containment

Key Benefit: Portability

- Virtual environments are hardware-independent
- Enables easy migration across systems
- Simplifies workload movement
- Supports disaster recovery strategies
- Enables consistent deployment environments
- Improves infrastructure flexibility
- Reduces vendor dependency
- Enhances operational mobility

Virtualization Role in Cloud Operations

- Enables on-demand infrastructure availability
- Supports rapid provisioning of resources
- Simplifies infrastructure automation
- Enhances scalability capability
- Improves service availability
- Enables efficient resource pooling
- Supports continuous service delivery
- Forms backbone of cloud services

Summary of the Lecture

- Virtualization abstracts physical hardware
- Enables efficient resource utilization
- Essential for cloud infrastructure flexibility
- Improves operational efficiency
- Provides strong workload isolation
- Enhances portability and mobility
- Supports scalable cloud operations
- Forms the foundation of cloud environments

Faculty-curated videos, NPTEL, Coursera, LinkedIn, or other relevant learning resources

- Cloud Computing: https://onlinecourses.nptel.ac.in/noc21_cs14/preview
- Cloud Computing: https://onlinecourses.nptel.ac.in/noc26_cs55/preview
- Google Cloud Computing Foundations: https://onlinecourses.nptel.ac.in/noc20_cs55/preview
- Google Cloud Computing Foundations: https://onlinecourses.nptel.ac.in/noc24_cs131/preview

Next Lecture

Types of virtualization (full, paravirtualization, hardware assisted), Type 1 vs Type 2 hypervisors, virtualization architecture

Quiz/ FAQ's

- Q1. Why is virtualization required in cloud environments?
- Solution: It enables efficient resource sharing, scalability, and rapid provisioning.
- Q2. How does virtualization improve hardware utilization?
- Solution: By allowing multiple workloads to share the same physical resources.
- Q3. What problem does isolation solve?
- Solution: It prevents failures or security issues in one environment from affecting others.
- Q4. Why is portability important in virtualization?
- Solution: It enables easy migration and recovery across different infrastructures.

References/ Articles/ Videos

- https://www.redhat.com/en/topics/virtualization
- https://www.vmware.com/topics/glossary/content/virtualization
- https://aws.amazon.com/what-is/virtualization/
- https://cloud.google.com/learn/what-is-virtualization
- https://www.ibm.com/topics/virtualization
- https://learn.microsoft.com/en-us/virtualization
- https://www.cisco.com/learn/virtualization
- https://www.digitalocean.com/community/tutorials

Class-Wise Feedback

Thank You

For queries
Email: sanjay.e18823@cumail.in

Academic Session 2025-26

EVEN Semester Jan-Jun 2026

UNIVERSITY INSTITUTE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BTECH CSE – CS 201

SEMESTER: 6TH

CLOUD COMPUTING

(23CSH-307)

Unit No. 1 **Chapter No. 3** **Lecture No. 9**

Topic : Types of virtualization (full,para-virtualization, hardware-assisted), Type 1 vs Type 2 hypervisors, virtualization architecture

Sanjay Tiwari, E18823 **Senior Technical Trainer**

RECAP

- Virtualization abstracts physical hardware into logical resources.
- It enables efficient utilization of compute, storage, and network resources.
- Virtualization is a core enabler of cloud infrastructure flexibility.
- It improves operational efficiency through centralized management.
- Strong isolation ensures secure and stable workload execution.
- Virtualized environments enhance application portability and mobility.
- Resources can be scaled dynamically using virtualization techniques.
- Virtualization supports multi-tenant cloud architectures.
- It allows rapid provisioning and recovery of workloads.
- Virtualization forms the technological foundation of cloud environments.

Learning Objectives

- Understand different types of virtualization
- Distinguish between Type 1 and Type 2 hypervisors
- Explain virtualization architecture
- Identify suitable virtualization approaches

Overview of Virtualization Types

- Virtualization enables multiple environments on one machine
- Different techniques manage hardware access
- Each type balances performance and complexity
- Virtualization type affects system efficiency
- Choice depends on workload requirements
- Compatibility varies across virtualization methods
- Performance overhead differs by approach
- Architecture determines isolation and control

Full Virtualization

- Provides complete hardware abstraction
- Guest OS runs without modification
- Hypervisor emulates underlying hardware
- Supports unmodified operating systems
- Ensures strong isolation between environments
- Slight performance overhead due to emulation
- Widely adopted in enterprise systems
- Suitable for diverse OS environments

Paravirtualization

- Guest OS is modified to interact with hypervisor
- Reduces need for hardware emulation
- Improves performance efficiency
- Requires OS-level awareness
- Enables faster system calls
- Lower overhead compared to full virtualization
- Compatibility depends on OS support
- Used where performance is critical

Hardware-Assisted Virtualization

- Uses CPU virtualization extensions
- Hardware directly supports virtualization tasks
- Reduces hypervisor complexity
- Improves performance and efficiency
- Allows unmodified guest operating systems
- Enhances isolation and security
- Minimizes virtualization overhead
- Common in modern virtualization platforms

Hypervisor: Core Concept

- Acts as virtualization control layer
- Manages access to physical resources
- Allocates CPU, memory, and storage
- Ensures isolation between virtual machines
- Controls VM lifecycle operations
- Enables multiple OS instances
- Optimizes resource scheduling
- Forms backbone of virtualization

Type 1 Hypervisor (Bare-Metal)

- Runs directly on physical hardware
- No host operating system required
- Provides high performance
- Offers strong isolation
- Preferred for enterprise environments
- Lower latency and overhead
- Enhanced security and stability
- Suitable for production workloads

Type 2 Hypervisor (Hosted)

- Runs on top of a host OS
- Easier to install and manage
- Suitable for development and testing
- Depends on host OS performance
- Higher overhead compared to Type 1
- Limited scalability
- Weaker isolation than Type 1
- Common in personal or lab environments

Type 1 vs Type 2 Hypervisors

Feature	**Type 1**	**Type 2**
Placement	Runs directly on hardware	Runs on host OS
Host OS	Not required	Required
Performance	High	Lower
Overhead	Minimal	Higher
Security	More secure	Less secure
Use Case	Enterprise / Production	Testing / Learning

Virtualization Architecture

- Physical hardware forms base layer
- Hypervisor sits above hardware
- Virtual machines run on hypervisor
- Each VM includes guest OS
- Virtual resources mapped to physical ones
- Isolation enforced by hypervisor
- Management layer controls operations
- Architecture supports scalability

Components of Virtualization Architecture

- Physical compute, memory, and storage
- Hypervisor management layer
- Virtual machines as execution units
- Guest operating systems
- Virtualized network interfaces
- Virtual storage controllers
- Resource scheduling mechanisms
- Monitoring and management tools

Summary of the Lecture

- Virtualization has multiple implementation types
- Full virtualization supports unmodified OS
- Paravirtualization improves performance
- Hardware assistance enhances efficiency
- Hypervisors manage virtual environments
- Type 1 and Type 2 serve different use cases
- Architecture defines control and isolation
- Proper choice improves system efficiency

Next Lecture

Architectural comparison of containers and virtual machines, performance analysis, use case selection criteria

Faculty-curated videos, NPTEL, Coursera, LinkedIn, or other relevant learning resources

- Cloud Computing: https://onlinecourses.nptel.ac.in/noc21_cs14/preview
- Cloud Computing: https://onlinecourses.nptel.ac.in/noc26_cs55/preview
- Google Cloud Computing Foundations: https://onlinecourses.nptel.ac.in/noc20_cs55/preview
- Google Cloud Computing Foundations: https://onlinecourses.nptel.ac.in/noc24_cs131/preview

Quiz/ FAQ's

Q1. Why are different virtualization types required?

Solution: Different workloads require trade-offs between performance, compatibility, and complexity.

Q2. Which virtualization type offers best performance?

Solution: Hardware-assisted virtualization provides optimal performance.

Q3. Why modify OS in paravirtualization?

Solution: To reduce overhead and improve communication with the hypervisor.

Q4. What is the main advantage of full virtualization?

Solution: It supports unmodified guest operating systems.

Q5. Why is Type 1 hypervisor preferred in enterprises?
Solution: It provides better performance, security, and stability.

References/ Articles/ Videos

- https://www.redhat.com/en/topics/virtualization
- https://www.vmware.com/topics/glossary/content/hypervisor
- https://aws.amazon.com/what-is/hypervisor/
- https://cloud.google.com/learn/what-is-a-hypervisor
- https://www.ibm.com/topics/virtualization
- https://learn.microsoft.com/en-us/virtualization
- https://www.cisco.com/learn/virtualization
- https://www.digitalocean.com/community/tutorials

Class-Wise Feedback

Thank You

For queries
Email: sanjay.e18823@cumail.in

Academic Session 2025-26

EVEN Semester Jan-Jun 2026

UNIVERSITY INSTITUTE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BTECH CSE – CS 201

SEMESTER: 6TH

CLOUD COMPUTING

(23CSH-307)

Unit No. 1 **Chapter No. 3** **Lecture No. 10**

Topic : Architectural comparison of containers and virtual machines, performance analysis, use case selection criteria

Sanjay Tiwari, E18823 **Senior Technical Trainer**

Recap

- Virtualization can be implemented using multiple architectural approaches.
- Full virtualization allows unmodified operating systems to run on virtual machines.
- Paravirtualization improves performance by modifying the guest operating system.
- Hardware-assisted virtualization enhances efficiency using CPU support features.
- Hypervisors are responsible for managing and controlling virtual environments.
- Type 1 hypervisors run directly on hardware for higher performance.
- Type 2 hypervisors run on host operating systems for ease of use.
- Virtualization architecture determines control, isolation, and resource allocation.
- Different use cases require different virtualization approaches.
- Selecting the appropriate virtualization type improves overall system efficiency.

Learning Objectives

- Understand container and virtual machine architectures
- Compare performance characteristics of containers and VMs
- Analyze architectural differences between containers and VMs
- Identify appropriate use cases for each technology

Overview of Containers and Virtual Machines

- Both enable application isolation
- Both support multiple workloads on one system
- Differ in abstraction level
- Differ in resource management approach
- Used to improve deployment efficiency
- Support modern application environments
- Address scalability and portability needs
- Chosen based on workload requirements

Container Architecture

- Applications run in isolated user spaces
- Shares host operating system kernel
- No separate guest operating system
- Lightweight runtime environment
- Faster startup and shutdown
- Minimal resource overhead
- Uses OS-level isolation mechanisms
- Designed for microservice architectures

Virtual Machine Architecture

- Each VM includes a full guest OS
- Hypervisor manages hardware access
- Strong isolation at hardware level
- Independent operating system per VM
- Higher resource consumption
- Slower startup time
- Supports heterogeneous OS environments
- Mature and stable architecture

Architectural Comparison: Containers vs VMs

- Containers share host OS kernel
- VMs run separate guest operating systems
- Containers are lightweight
- VMs are heavier in resource usage
- Containers start quickly
- VMs have longer boot times
- Containers rely on OS isolation
- VMs rely on hardware virtualization

Performance Analysis: Containers

- Near-native performance
- Low CPU and memory overhead
- Faster application startup
- Efficient resource utilization
- High density of workloads possible
- Ideal for short-lived processes
- Lower latency communication
- Performance tied to host OS

Performance Analysis: Virtual Machines

- Higher overhead due to guest OS
- Increased memory and storage usage
- Slower boot and provisioning time
- Strong performance isolation
- Suitable for long-running workloads
- Stable performance under load
- Better fault isolation
- Performance depends on hypervisor

Use Case Selection: Containers

- Microservices-based applications
- Continuous integration and deployment
- Stateless or loosely coupled services
- Rapid development and testing
- High scalability requirements
- Resource-efficient environments
- Fast application startup needed
- Platform consistency required

Use Case Selection: Virtual Machines

- Legacy or monolithic applications
- Applications requiring full OS control
- Strong isolation and security needs
- Multiple operating systems required
- Long-running stable workloads
- Compliance-driven environments
- Traditional enterprise applications
- Predictable resource allocation

Decision Factors for Selection

- Application architecture style
- Isolation and security requirements
- Performance and startup needs
- Resource availability
- Operational complexity tolerance
- Compatibility with existing systems
- Scalability expectations
- Management and maintenance effort

Summary of the Lecture

- Containers and VMs enable workload isolation
- Architecture differs significantly
- Containers are lightweight and fast
- VMs provide stronger isolation
- Performance varies by design
- Use cases determine suitability
- Containers suit modern applications
- VMs suit traditional workloads

Faculty-curated videos, NPTEL, Coursera, LinkedIn, or other relevant learning resources

- Cloud Computing: https://onlinecourses.nptel.ac.in/noc21_cs14/preview
- Cloud Computing: https://onlinecourses.nptel.ac.in/noc26_cs55/preview
- Google Cloud Computing Foundations: https://onlinecourses.nptel.ac.in/noc20_cs55/preview
- Google Cloud Computing Foundations: https://onlinecourses.nptel.ac.in/noc24_cs131/preview

Next Lecture

Concept of redundancy, failover mechanisms, active-active vs. active-passive configurations, designing for failure.

Quiz/ FAQ's

- Q1. Are containers faster than virtual machines?
- Solution: Yes, containers generally offer better performance due to lower overhead.
- Q2. Do containers replace virtual machines?
- Solution: No, both coexist and serve different workload requirements.
- Q3. Why do VMs consume more resources?
- Solution: Each VM runs a full guest operating system.
- Q4. Which offers stronger isolation?
- Solution: Virtual machines provide stronger isolation than containers.
- Q5. Are containers suitable for all applications?
- Solution: No, applications needing full OS control may require VMs.

References/ Articles/ Videos

- https://www.redhat.com/en/topics/containers/containers-vs-vms
- https://www.ibm.com/topics/containers-vs-vms
- https://aws.amazon.com/compare/the-difference-between-containers-and-virtual-machines/
- https://cloud.google.com/learn/containers-vs-vms
- https://www.vmware.com/topics/glossary/content/container-vs-virtual-machine
- https://learn.microsoft.com/en-us/virtualization/windowscontainers/about/containers-vs-vms
- https://www.docker.com/resources/what-container
- https://kubernetes.io/docs/concepts/overview/

Class-Wise Feedback

Thank You

For queries
Email: sanjay.e18823@cumail.in